

Rail

Solution

Way : Ad Hoc

Query complexity: $3(N-1)$

Time complexity : $O(N \log N)$

** This problem can be solved by the following steps :

First, we know that station 0 is C type, and its location. We can query all the other stations' distances from station 0, we call this : `dis[0][i]`

Second, we sort all the `dis[0][i]`, and obviously, the station x with the shortest distance `dis[0][x]` must be the first D type location after station 0.

Third, we process each station one by one according to their shortest distance with station 0 (that is, the order obtained in second step). For each station processed, we determine its type and location immediately as follows:

3.1 First, we maintain the information (`location,id`) of the leftmost C type and the rightmost D type as the algorithm proceeds.

3.2 To process the current station k, we use two queries, `query(k, leftmost C type)` as `dis[k][L]`, `query(k, rightmost D type)` as `dis[k][R]`. And we also have `dis[0][k]`. By some observations, we know that either `dis[k][L]` or `dis[k][R]` is achieved with a 'direct' route (without moving forth and back).

For example, we have only 4 cases to consider :

a. `dis[k][L]` is a direct route

```
(   )   )  
L    k   R
```

b. `dis[k][L]` is a direct route

```
(           )   )  
L             R    k
```

c. `dis[k][R]` is a direct route

```
( (           )  
k  L             R
```

d. `dis[k][R]` is a direct route

```
( (   )  
L    k   R
```

By careful analysis (some if/else conditions), we can get the answer. Sometimes, we may need extra information to resolve the four cases, where we can check with `dis[0][k]`.

Take case (a) for example :

a. `dis[k][L]` is a direct route

$$\begin{pmatrix} & & & \end{pmatrix}$$

$$L \quad k \quad R$$

the location of k might be $L + \text{dis}[k][L]$, Then use this location, we need to check if `dis[k][R]` is reasonable or not.

a.1

$$\begin{pmatrix} (&) & \end{pmatrix}$$

$$L \quad p \quad k \quad R$$

There might be some p(type C) between L and k, So the `dis[k][R]` is `dis[p][R]+dis[p][k]`, we need to check whether p exists or not. If p doesn't exist, then case (a) might be wrong, then try cases (b),(c), and (d) until we find the answer.

Solution Sketch for Wall

Overview

We'll simplify the notion of the problem as follow:

Initially we have an array of length n where the value at each index is 0, and we are to process k queries in order. We will denote the value at index x as $A[x]$. There are two kinds of operations:

- Minimize(l, r, t): For all indices x between $[l, r]$, the value become $\min(A[x], t)$
- Maximize(l, r, t): For all indices x between $[l, r]$, the value become $\max(A[x], t)$

The **minimize** operation corresponds to the original **remove** operation;

the **maximize** operation corresponds to the original **add** operation.

Trivial solution - $O(nk)$

A trivial solution to this problem is to use an array of length n to maintain the current height at each index, and for each query perform a linear update in $O(n)$. This solution runs in $O(nk)$.

Segment Tree - $O((n + k) \lg n)$

One crucial observation is that for a specific position, any cascade of operations applied on it could be simplified to one **minimize** operation and one **maximize** operation.

For example, applying (here we omit the parameter l, r as we consider operations solely on a specific index):

- minimize(9), minimize(8), minimize(7) $\Rightarrow$ minimize(7), maximize($-\infty$)
- minimize(3), maximize(7), minimize(4) $\Rightarrow$ minimize(4), maximize(4)

In general, this observation could be verified by simple induction.

With this observation we could now improve the update time. We instead use a segment tree to maintain *the final operations applied on an interval*.

A segment tree is basically a binary tree, where each node contains the following information:

```

class Node {
    Node *left, *right; // children nodes
    int from, to;        // corresponding interval [from, to]
    int opmin, opmax;    // min/max operation applied on interval
};

```

An minimize operation would look something like:

```

void Node::minimize(int l, int r, int t) {
    if( from==l && to==r ) { // applied on full segment
        opmin = min(opmin, t);
        opmax = min(opmin, opmax);
    } else {
        // pass down previously applied operation
        left->minimize( left->from, left->to, opmin );
        left->maximize( left->from, left->to, opmax );
        right->minimize( right->from, right->to, opmin );
        right->maximize( right->from, right->to, opmax );
        // recursively apply current minimize operation
        if( r<=left->to ) left->minimize( l, r, t );
        else if( l>=right->from ) right->minimize( l, r, t );
        else {
            left->minimize( l, left->to, t );
            right->minimize( right->from, r, t );
        }
    }
}

```

A single round of minimize operation runs in $O(\lg n)$, likewise for maximize operations. Note that in the snippet above, we use the well-known lazy-update technique to maintain the operations applied at one position. i.e. the message is passed down to children nodes *only when necessary*, that is, when new operations are imposed on only part of the segment, so previous operations should be passed down first.

After all queries are processed, we simply scan each position and check (with $\lg n$ complexity) the operations applied on it, obtaining the final value at the position.

The overall running time is therefore $O((n + k) \lg n)$.

It is possible to optimize further so that the overall running time become $O(n + k \lg k)$, it is however entirely not necessary for this problem.

Game Solution

Let E_{yes} be the set of edges about which the contestant has answered “yes” (connected), E_{no} the set of edges about which contestant has answered “no”, and E_{maybe} the rest of the edges, whose statuses are not yet determined. Also, let $G = (V, E_{\text{yes}})$ and $H = (V, E_{\text{yes}} \cup E_{\text{maybe}})$. G is the graph you get by assuming that every edge in E_{maybe} are not connected, while H is the graph you get by assuming that all edges in E_{maybe} are connected.

Initially, G is empty and thus not connected, while H is connected. In order not to reveal any clue to the judge, the contestant should maintain the invariant: G should always be disconnected, while H should always be connected.

There are several possible ways to maintain the invariant.

An $O(n^4)$ solution

When asked by the judge whether an edge $e = (u, v)$ is connected, answer “no” if and only if e is part of a cycle in H . One can see that this does not change the connectivity of G and H .

To decide whether e forms a circle, one can perform a depth-first search to find out whether there is a path from u to v in $(V, E_{\text{yes}} \cup E_{\text{maybe}} - (u, v))$. This is an $O(n^2)$ operation. As there are $O(n^2)$ edges, the total running time is $O(n^4)$.

In other words, we answer “yes” if and only if e is a bridge in H .

An $O(n^2)$ solution

Given a vertex v , let $D(v)$ be the connected component v belongs to in G . We maintain two data structures:

1. R is a table mapping each v to a representative of $D(v)$.
2. S is a symmetric matrix indexed by V . For u and v in V , if $R(u) \neq R(v)$, $S(R(u), R(v))$ is the number of edges, in E_{maybe} , that connects $D(u)$ and $D(v)$.

The contestant answers “yes” to query (u, v) if and only if $S(R(u), R(v)) = 1$.

R can be implemented as a disjoint-set linked list. Each disjoint set is represented by a linked list of its elements, and the representative is the one at the head. Each element has a pointer to its representative. To unite two sets we connect the lists, and update the pointers. An union takes $O(n)$ time and a find takes $O(1)$ time.

As for S , initially $S(u, v) = 1$ unless $u = v$. Whenever the judge asks about (u, v) , S is updated as follows.

1. If the contestant answers “no”, we decrement $S(R(u), R(v))$ by 1.
2. If the contestant answers “yes”, let w be the representative after uniting $D(u)$ and $D(v)$. For each x that is a representative of some connected component, both $S(w, x)$ and $S(x, w)$ are updated to $S(R(u), x) + S(R(v), x)$.

There can be at most $n - 1$ unions, thus the total time spent on union is $O(n^2)$. An update of S requires $O(1)$ time for a “no” response, and $O(n)$ time for a “yes” response. Since the graph G is a tree, we respond “yes” exactly $n - 1$ times. Thus the time spent on updating S is also $O(n^2)$. We thus have an $O(n^2)$ algorithm.

An One-Liner $O(n^2)$ Algorithm

There is a surprising one-line $O(n^2)$ algorithm:

```
#include "game.h"

void initialize(int n) {
    // DO NOTHING!
}

int c[1500];
int hasEdge(int u, int v) {
    return ++c[u > v ? u : v] == (u > v ? u : v);
}
```

To understand the algorithm, imagine that we partition the set of all the possible edges into $E_1, E_2, \dots, E_{n-1}$, with $E_i = \{(i, j) \mid i > j\}$. Each E_i has exactly i possible edges. The algorithm above answers “yes” to (u, v) (where $u > v$) if it is the last edge in E_u that is queried.

To see how it works, consider the last query. Denote the queried edge by e , and the graph $G = (V, E_{\text{yes}} - e)$. The contestant wins if G is disconnected, while $G + e$ is connected.

- G is disconnected, since it contains only $n - 2$ edges.
- $G + e$ is connected, since it contains $n - 1$ edges, and there is no cycle in $G + e$. One can see that there is no cycle since, in each E_i , we answer yes to only one edge. Formally, if there is a cycle C in $G + e$, considering the node u in C with largest id, E_u must have exactly one edge in $G + e$. But u has two neighbors in C with smaller ids, a contradiction.

Gondola

Solutions

Overview

This is an easy problem. Even though it might seem that there are three separate tasks (as indicated by the grouping of subtasks), the task is

actually incremental. The three parts (check whether there is a solution – find one solution – count all solutions) are closely related.

Subtasks 1-3

In subtask 1, once we see the first gondola (i.e., `inputSeq[0]`), the rest is uniquely determined. We just need to iterate through the sequence and check whether everything matches.

- `N = len(sequence)`
- for `n:=1..N`: if `sequence[n] % N != (sequence[0]+n) % N`: return `False`
- return `True`

The same code will actually solve subtask 2.

The pseudocode for the general subtask 3 is:

- If there exist one of the original gondolas: check whether the other original gondolas are in the expected places, if not, return `false`.
- Return `true` if all the values are distinct, `false` otherwise.

Subtasks 4-6

A simple solution for subtask 4: if the largest number in the sequence is `n`, terminate, otherwise output the only missing number and terminate.

Solutions for subtasks 5 and 6 share the same idea, the difference is that subtask 4 allows its inefficient implementations. There are many possible solutions. Here is one of them.

- Collect all non-original gondolas. For each of them determine the original gondola it replaced.
- Sort these records according to the new gondola number.
- In sorted order, replace the original gondolas by new ones until the expected numbers are reached.

Note that the case where no original gondolas are present may require special attention – not just for the contestants, but also in the grader for these subtasks.

Subtasks 7-10

Counting the repair sequences directly is hard. One way of doing it is by asking the question: **How many repair sequences start with gondola x being replaced?** for each x . Each choice of x leads us to a new state with one fewer gondolas to replace.

Using this idea we can now solve subtask 8 by dynamic programming: for each admissible state of the lift we compute the number of ways in which it can be solved.

Subtasks 9 and 10 require one additional insight. (This is, probably, the only tricky part of this problem, and the insight needed is not too hard.)

Instead of looking at the old gondola that is being removed, we will simply look at the new gondola that is being added. How many different

options do we have for its place? If the new gondola is present in the final sequence, its place is uniquely determined. Otherwise, the number of places where

this new gondola can be added is simply the number of places that end up having a gondola with a larger number.

Additionally, we need to multiply the result by n if none of the original gondolas is present. (All n cyclic rotations of the original sequence are now possible,

and the repair sequences for different rotations are necessarily distinct.) We outline the algorithm as follows:

- Run the algorithm for subtasks 1-3 to check whether the input sequence is valid or not.
- If valid, for each replaced gondola we find the original gondola it ultimately replaced, and we sort these records according to the new gondola number.
- The total number of possibilities can now be computed by multiplying the number of possible locations for each gondola between $n + 1$ and the largest replaced gondola number present. Note the multiplicative operation is modular here.
- Finally, multiply by n if all original gondolas are replaced.

Friends

6 solutions to 6 subtasks

	Solution1	Solution2	Solution3	Solution4	Solution5	Solution6
Subtask1	AC	WA	WA	WA	WA	AC
Subtask2	TLE	AC	WA	WA	WA	AC
Subtask3	TLE	WA	AC	WA	WA	AC
Subtask4	TLE	WA	WA	AC	WA	AC
Subtask5	TLE	WA	WA	WA	AC	AC
Subtask6	TLE	WA	WA	WA	WA	AC

Table 1-1: The result of each solution applying to each subtask.
Note: AC=Accepted, WA=Wrong Answer, TLE=Time Limit Exceeded.

Solution1

In subtask1, N is at most 10. So just apply backtracking for every person – to chose or not to chose. The complexity is $O(N * 2^N)$, Accepted.
The sizes of N in other subtasks are too large to apply this solution, resulting in Time Limit Exceeded with this complexity.

Solution2

In subtask2, there're all 'MyFriendsAreYourFriends' relations, forming a graph with no edge. That is equivalent to chose all persons, with complexity of $O(N)$.
For other subtasks, there're not only this kind of relations, so this solution does not work and will result in Wrong Answer.

Solution3

In subtask3, there're all 'WeAreYourFriends' relations, forming a complete graph. Since every pair of two persons is connected by an edge, the answer to this problem is equivalent to choose the maximum confidence among all people, with complexity of $O(N)$.
For other subtasks, there're not only this kind of relations, so this solution does not work and will result in Wrong Answer.

Solution4

In subtask4, all relations are 'IamYourFriend', forming a tree. So we apply the DP-in-tree method.

Define $dp[i][j]$ as the maximum sum for the i^{th} node with status j , where $j = 0$ stands for not choosing this node and $j = 1$ stands for choosing this node. Then:

(1) If the i^{th} node is leaf, then

- $dp[i][j] = 0$, for $j = 0$.
- $dp[i][j] = confidence[i]$, for $j = 1$.

(2) Otherwise,

- $dp[i][j] = \max(dp[k][0], dp[k][1])$, for $j = 0$ and for all k , where k is i 's child.
- $dp[i][j] = dp[k][0]$, for $j = 1$ and for all k , where k is i 's child.

The final answer is $\max(dp[root][0], dp[root][1])$, where root stands for the root of this tree.

For other subtasks, there're not only 'IamYourFriend' relations, so this solution will not work and will result in Wrong Answer.

Solution5

Since there're only 'MyFriendsAreYourFriends' and 'IamYourFriend' relations, the resulting graph contains no odd cycle. That is, we obtain a bipartite graph. With all confidence equals to 1, the problem becomes finding maximum independent set in a bipartite graph. As we know, a set is independent if and only if its complement is a vertex cover. If the complement of independent set is not a vertex cover, then there exists at least one edge with end points u and v , which is included in the independent set, conflicting with the definition of independent set. Trivially, a maximum independent set is the complement of minimum vertex cover.

According to Konig's theorem[1], in any bipartite graph, the number of edges in a maximum matching is equal to the number of vertices in a minimum vertex cover. Thus, we can apply the augmenting path algorithm to find out maximum cardinality matching in a bipartite graph, with complexity of $O(NE)$ or $O(\sqrt{N}E)$, depending on different implementations, where E is the number of edges.

Let k be the result of maximum cardinality bipartite matching, the answer to this problem equals to $N - k$, since maximum independent set is the complement

of minimum vertex cover.

As for partitioning the graph into bipartite, we apply *dfs* to mark out the odd points and the even points, and then put all odd points one side, even points the other side.

In subtask 2 and 4, the relations are ‘MyFriendsAreYourFriends’ and ‘IamYourFriend’. However, the confidence value in those two subtasks are not all equals to 1. As a result, this solution can only solve subtask5 correctly, and Wrong Answer for other subtasks.

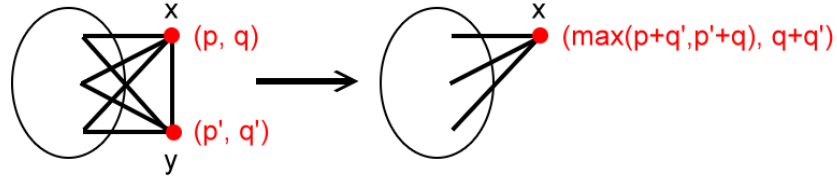
Solution6

By using Greedy method to eliminate each person in the reverse order of building process, we will finally get the (p, q) pair for the last person. The answer will be $\max(p, q)$ of the last person.

Here we briefly introduce this method. Initially, we maintain two values $p(x)$ and $q(x)$ for each person x , where $p(x) = \text{confidence}[x]$ and $q(x) = 0$. Physically, p stands for ‘choose’ and q stands for ‘not choose’.

To simplify the notation, we call p for $p(x)$, q for $q(x)$, p' for $p(y)$, q' for $q(y)$.

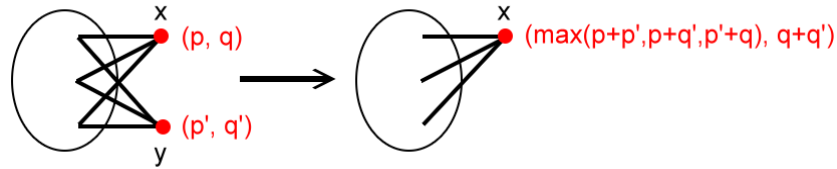
(1) WeAreYourFriends



To eliminate y , we can either choose x or chose y , or neither of both.

- (a) Choose x : $p = p + q'$.
- (b) Choose y : $p = p' + q$.
- (c) Neither: $q = q + q'$. So $p = \max(p + q', p' + q)$, $q = q + q'$.

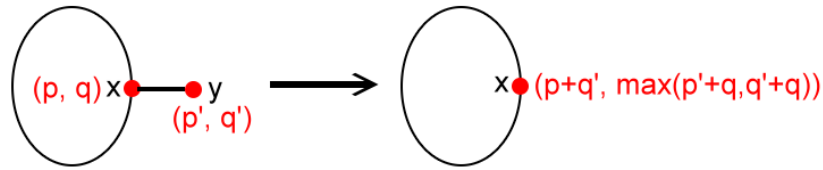
(2) MyFriendsAreYourFriends



To eliminate y , we can choose x , choose y or choose both, or neither of both.

- (a) Choose x : $p = p + q'$.
- (b) Choose y : $p = p' + q$.
- (c) Choose both: $p = p + p'$.
- (d) Neither: $q = q + q'$. So $p = \max(p + q', p' + q, p + p')$, $q = q + q'$.

(3) IamYourFriend



To eliminate y , we can choose x , or either choose y or choose neither.

- (a) Choose x : $p = p + q'$.
- (b) Choose y : $q = p' + q$.
- (c) Neither: $q = q + q'$. So $p = p + q'$, $q = \max(p' + q, q + q')$.

The complexity of this scheme is $O(N)$. This method is capable of solving all subtasks.

Reference

[1] *König's theorem*
[http://en.wikipedia.org/wiki/K%C3%B6nig%27s_theorem_\(graph_theory\)](http://en.wikipedia.org/wiki/K%C3%B6nig%27s_theorem_(graph_theory))

Solution for the Holiday Task

For ease of description, we will first describe a simple $O(n \log n)$ -time solution for the special case of the starting city *start* being the one with the index 0 for any fixed d . That is, $start = 0$ and d is a given number. Then we extend this solution to build a table of solutions in $O(n \log^2 n)$ time for all possible values of d . Finally we describe how to extend this solution to solve the general case of an arbitrary *start* with the same asymptotic time complexity.

Simple solution for $start = 0$ and a given fixed d

Without loss of generality, assume we start at the leftmost city and move right. It is easy to see that we only need to move right and there is no need to move left at any time. Assume in an optimal solution, city *right* is the rightmost city we will travel to. Then we can visit up to $d - right$ cities among the cities with labels $0, 1, \dots, right$. In order for the solution to be optimal, we want to visit the $d - right$ cities with the largest number of attractions. That is, if we sort the cities with labels $0, 1, \dots, right$ using the number of attractions as their keys, then we want to know the sum of the $d - right$ largest number of attractions.

Segment tree We use a data structure called segment tree for this part though it may appear this data structure is not needed to solve this very special case. However, it will be clear why this data structure is used in the solution to the intermediate case. The segment tree data structure has been used in previous IOI contests including 2001 Baltic OI. The segment tree has many variations. We will use the following one. A segment tree is a rooted complete binary tree with leaves carrying a flag indicating whether this leaf is active or not, and a value. For each internal node v , it keeps the sum of the values of all of the active leaves in the subtree rooted at v . Each internal node also maintains the number of currently active leaves in the subtree rooted at v .

Assume the segment tree has n leaves. Note that we need to add dummy leaves if n is not a power of 2. Further assume the values of the leaves, active or in-active, are in non-increasing order from left to right. To maintain this data structure, it takes $O(\log n)$ time to turn on or off any leaf. It also takes $O(\log n)$ time to find out the sum of the values in the largest x active leaves for any given x . A side note is when x is more than the number of active leaves, then we simply output the sum of the values of all active leaves.

Algorithm Initially, we sort the cities using their number of attractions as keys in non-increasing order. Then in this order, we place them as leaves in the segment tree from left to right with all leaves in-active. The number of attractions are now the values of the leaves. The initialization phase takes $O(n \log n)$ time. We turn on a leaf when it is being move to during the search of our solution. We iterate on all possible values of the rightmost city we can move to. Hence it takes a total of $O(n \log n)$ time to find a solution for this easy special case.

Intermediate solution for $start = 0$ and all possible values of d

Now we describe how to solve this intermediate case. In our previous solution, we can find the maximum number of attractions we can visit given any d . Let $f(d)$ be the label of the city we move to in d days so that the maximum number of attractions can be found. Note that $f(d)$ may not be unique. In the case of multiple ones, we pick the one with the smallest label. We now want to build a table for all possible values of d . The idea is to use recursive divide-and-conquer approach. Let M be the maximum number of d . For ease of description, let M be a power of 2. To compute the solutions for $f(1), f(2), \dots, f(M)$ we first find $f(M/2)$ using our previous algorithm by iterating through all cities from 0 to n and then recursively compute $f(1), f(2), \dots, f(M/2 - 1)$ in one branch by considering only cities from 0 to $f(M/2)$, and $f(M/2 + 1), f(M/2 + 2), \dots, f(M)$ in the other branch by considering only cities from $f(M/2)$ to n . In the branch of computing $f(1), f(2), \dots, f(M/2 - 1)$, we first compute $f(M/4)$ among cities 0 to $f(M/2)$. In general, the total amount of time spent in each level of recursive calls takes a total of $O(n \log n)$. There are a total of $O(\log n)$ levels. Hence the overall time complexity is $O(n \log^2 n)$.

In solving this intermediate case, it is now clear how the segment tree is useful. First we only need to do the initialization once. Secondly, we can easily turn on or off a leaf to accommodate the fact that the cities that we pay attention to in each level of recursion. For example, only half of the leaves are active during the second level of recursion.

General solution for an arbitrary value of $start$

Now we are ready to show the general solution. Observe the fact that when the value of $start$ is arbitrary, then the solution can be found in either one of the following 2 cases. We first move right to a city, then move left from this point, and then finally stop at a city left of $start$. Or we first move left to a

city, then move right from this point, and then finally stop at a city right of *start*. That is, we only change the direction once. Without loss of generality, we assume the first scenario. We first use the immediately solution to find $f(t)$ for all possible values t . Then we also use the immediately solution to find $g(t)$ for all possible values of t where $g(t)$ is the city we will stop in an optimal solution if we only move left and the starting city is $start - 1$. We first iterate on d_0 the days we want to spend on moving and visiting cities to the right of, and including, *start*. Using the solution to the intermediate case, we know $f(d_0)$. Then we know we can spend $d - d_0 - (f(d_0) - start) - 1$ days on the cities to the left of *start*. Hence the overall time complexity is $O(n \log^2 n)$.